

RapidGrid — demo runbook

A GeoAgentic framework for emergency vehicle movement in Bengaluru. This is the script for demonstrating it end to end: every role, every API call, and the evidence that nothing on screen is invented.

The one-line thesis: the nearest hospital is often the wrong one, and this system can say exactly why it drove past it.

0. Before you start (10 minutes)

Start the stack

```
cd backend && ./venv/Scripts/python.exe -m uvicorn main:app --reload --port 8000
```

```
cd frontend && npm run dev
```

Frontend on `http://localhost:5173`, backend on `http://localhost:8000`. The frontend calls the backend through Vite's proxy (`/api`, `/health`, `/ws`), so there is no hardcoded backend URL anywhere in the client.

Preflight — run this every single time

```
cd backend && ./venv/Scripts/python.exe scripts/preflight.py
```

Checks all three API keys, both routing tiers, the road graph, the hospital capability table, and OpenAPI schema generation in about 15 seconds. **Green before you present.** A warning on Google Routes is survivable (the offline router covers it); a FAIL is not.

One-time, if the road graph is missing

```
cd backend && ./venv/Scripts/python.exe scripts/build_road_graph.py
```

Pulls Bengaluru's drivable network from OpenStreetMap and writes a compact 1.6 MB graph: **112,725 nodes, 178,089 directed edges.**

Reset to a clean slate

```
curl -s -X POST http://127.0.0.1:8000/api/incidents/clear && curl -s -X POST http://127.0.0.1:8000/api/city/clear
```

Open four browser tabs

Tab	URL	Role
1	localhost:5173/citizen	Caller (Ananya Sharma, +91 98765 43210)
2	localhost:5173/dispatcher	City dispatch
3	localhost:5173/driver	Field unit (AMB-UNIT-04)
4	localhost:5173/hospital	Emergency department

Arrange them so you can switch quickly. Tab 2 is the one judges will look at longest.

Say this early: sign-in is deliberately open — picking a role grants it. The login screen states this itself. It is a demo affordance, not a security claim, and naming it before anyone asks buys you credibility for everything that follows.

1. The flow (about 6 minutes)

Beat 1 — the citizen raises an emergency (45s)

Tab 1. Press the SOS button, choose **Medical**, and type something real:

My father collapsed with severe chest pain and is barely conscious

Press **Send emergency report**.

Point out: the progress steps name what the system is actually doing — reading the report, assessing severity, checking traffic, ranking hospitals, routing. There is no fake telemetry. An earlier build cycled `SIGNAL STRENGTH: EXCELLENT / ACCURACY: 3 METERS` ; that was theatre and it is gone.

Behind it:

```
curl -s -X POST http://127.0.0.1:8000/api/incidents -H "Content-Type: application/json" -d '{"citizen_text":"My father collapsed with s
```

Returns `202 Accepted` immediately with a `poll_url` — the agents run in the background rather than blocking the caller.

Beat 2 — triage, and why it is not a black box (30s)

The call is classified **cardiac, severity 0.99, P1 CRITICAL**.

Show that the classifier is transparent and auditable, not a claimed ML model:

```
cd backend && ./venv/Scripts/python.exe -c "import sys; sys.path.insert(0, '.'); from agents.triage import classify; r=classify('Two-whe
```

Say this: it is a weighted-keyword model and we call it that. It reports low confidence when the evidence is thin — “someone fainted at the bus stop” comes back at 0.38, not 0.88 — because over-confidence is how these systems mislead people.

Beat 3 — the dispatch console (90s) ← *the important one*

Tab 2. The call is in the queue with a full plan. Walk through, in order:

- a) The ranked destinations.** Five hospitals, each with a real road ETA and a **VERIFIED** or **SIMULATED** provenance badge. Note that a cardiac call is ranked against cath-lab capability, and that oncology, maternity and day-surgery units were excluded from the candidate list entirely.
- b) The trade-off card.** Two columns — selected vs runner-up — showing the factor each one wins on and the road-minute cost. This is a genuine counterfactual: the system states which hospital it rejected and why.
- c) Time to definitive treatment.** The stacked bar breaks the clock into dispatch → drive to patient → on scene → transport → in-hospital workup → secondary transfer, against the 90-minute door-to-balloon target.

Say this: minutes on the road are a proxy. The outcome is time to treatment. A nearer hospital without a cath lab does not cost a few minutes — it costs a whole second journey, because the patient has to be transferred onward. That is why driving further can arrive sooner.

Prove it live:

```
cd backend && ./venv/Scripts/python.exe -c "
import sys; sys.path.insert(0, '.')
from agents.clinical_outcome import compare
near = {'name': 'Local general hospital', 'capabilities': {'cardiology': 0.42}}
pci = {'name': 'Jayadeva Institute of Cardiovascular Sciences', 'capabilities': {'cardiology': 1.00}}
r = compare('cardiac', pci, near, 8*60, 16*60, 5*60)
print(r['verdict'])
print('PCI centre:', r['chosen']['total_minutes'], 'min | within target:', r['chosen']['within_target'])
print('nearest ER:', r['alternative']['total_minutes'], 'min | within target:', r['alternative']['within_target'])
"
```

Drove 11.0 min further to reach primary angioplasty 48 min sooner. PCI centre : 60.7 min
(within target) · nearest ER : 109.1 min (misses the 90 min target by 19)

d) Approve and dispatch. The status flips to **EN ROUTE** and the call propagates to the driver and hospital tabs.

Beat 4 — the field unit (45s)

Tab 3. The call appears. Open it.

Two-stage navigation: **Stage 1** drives the unit from its station hub to the patient (dashed line on the map), **Stage 2** carries the patient to the ER (solid line). Tap **I have reached the patient** to advance.

Show **Change destination** — if the assigned ER diverts, the crew picks another and the route plus the caller’s screen update immediately.

Beat 5 — the emergency department (30s)

Tab 4. The patient is inbound with a countdown, severity band and summary.

Flip **ICU capacity** to **On divert**. This is a real control, not a toggle for show — it removes the facility from the routing engine's candidate list for new calls.

```
curl -s -X PATCH http://127.0.0.1:8000/api/hospital/blr-005/resources -H "Content-Type: application/json" -d '{"icu_available": false}'
```

Beat 6 — chat across all four roles (45s)

Every role shares one incident channel. Type in **Tab 1** as the caller:

He is breathing but very weak.

It appears in the driver and hospital tabs within a second. Reply from **Tab 3**:

4 minutes out. Keep him on his side.

Point at the header: it reads **Live** when the WebSocket is connected and **Delayed** when it has dropped to HTTP polling. It never claims live when it is not.

Verify from the terminal:

```
curl -s http://127.0.0.1:8000/api/chat/INC-XXXX/messages
```

Beat 7 — the city is alive (75s) ← *the memorable one*

Tab 2, Disruptions panel. Type **Bellary Road** and select it.

Watch: the closure draws on the map, a red congestion bloom spreads onto the surrounding streets, and the counters show segments closed and displaced.

Say this: traffic on a closed road does not evaporate. It reappears on parallel streets, which congest, which slows *a different ambulance somewhere else in the city*. That second-order effect is the whole point.

Prove it with numbers:

```
cd backend && ./venv/Scripts/python.exe -c "
import sys, logging; sys.path.insert(0, '.'); logging.basicConfig(level=logging.WARNING)
from agents.local_router import local_router
from agents.city_state import city_state
local_router.load(); city_state.attach(local_router); city_state.set_hour(18)
P=((13.0350,77.5970),(12.9290,77.5878)); N=((13.0420,77.5860),(12.9980,77.5710)); F=((12.9698,77.7180),(12.9352,77.6245))
t=lambda x:(local_router.find_path(*x) or {}).get('travel_time_s',0)/60
b={k:t(v) for k,v in dict(p=P,n=N,f=F).items()}
bell=[e for e in local_router.find_path(*P)['edge_ids'] if 'Bellary' in local_router.road_names[local_router.edge_name[e]]]
r=city_state.close_road(bell,'Bellary Road'); a={k:t(v) for k,v in dict(p=P,n=N,f=F).items()}
print(f"closed {r['closed']} segments -> displaced onto {r['displaced']} neighbours")
for k,lab in [ ('p','used the closed road'), ('n','parallel corridor (untouched)'), ('f','across town') ]:
    print(f' {lab}<32}{b[k]:5.1f} -> {a[k]:5.1f} min ({a[k]-b[k]:+.1f})' )
"
```

```
closed 52 segments -> displaced onto 97 neighbours
  used the closed road          28.8 -> 33.9 min (+5.1)
  parallel corridor (untouched) 13.1 -> 16.0 min (+2.9)  <- never used the road
  across town                   24.4 -> 24.4 min (+0.0)  <- correctly unaffected
```

The middle row is the demo. A route that never touched the closure gets slower. A route across town does not, because displacement is local.

Also show time of day and weather in the same panel — the same route runs 16.2 min at 03:00, 28.8 at evening peak, 38.9 in the rain.

Ask the causal model to explain any segment:

```
EDGE=$(curl -s "http://127.0.0.1:8000/api/city/overlay?limit=5" | python -c "import sys,json;print(json.load(sys.stdin)['congestion']")[0]
```

It returns the causal chain for that segment — which mechanism slowed it and by how much:

```
{ "edge_id": 177923, "blocked": false,
  "causes": [
    { "cause": "time_of_day", "detail": "06:00 traffic profile", "multiplier": 1.02 },
    { "cause": "displacement", "detail": "traffic displaced from a nearby closure", "multiplier": 1.85 }
  ],
  "total_multiplier": 1.89, "provenance": "modelled" }
```

Then reset:

```
curl -s -X POST http://127.0.0.1:8000/api/city/clear
```

Beat 8 — break it on purpose (45s) ← *the flex*

Turn off the wi-fi. Or revoke the Google key. Raise another SOS.

The system keeps routing. The Google tier fails, an A* router falls back to the cached OpenStreetMap graph, and the plan is labelled **Degraded data – live routing was unavailable** with the confidence figure dropped accordingly.

Say this: most demos die without wi-fi. The failure mode we care about is not “no route” — it is a system that keeps producing confident-looking numbers when it has no idea. Watch what it says about itself instead.

```
curl -s http://127.0.0.1:8000/health | python -m json.tool
```

`routing.offline_graph_available`, `billable_api_calls`, `cache_hits`, `hit_rate` — plus the live city model state.

Beat 9 — what is real and what is not (30s)

Scroll to the bottom of the landing page. Two columns, side by side.

Say this: every system like this models something. Most do not tell you which parts. Ours draws modelled values differently everywhere they appear — hatched badges, dotted underlines — so you never have to ask.

2. Proving there is no hardcoding

Judges will suspect the numbers are staged. These are the fastest disproofs.

Change the location and the answer changes

```
curl -s -X POST http://127.0.0.1:8000/api/hospital/rank -H "Content-Type: application/json" -d '{"incident_location":{"lat":12.9698,"lon":77.6412,"emergency_type":"cardiac"}}
```

Run it again with `"emergency_type":"cardiac"` — a different ranking, because the weight table and the specialty gate change.

The same emergency at different times of day

Beat 7's script already shows 16.2 / 28.8 / 38.9 minutes for one route at 03:00, peak, and peak-with-rain.

The road graph is real OpenStreetMap data

```
cd backend && ./venv/Scripts/python.exe -c "
import sys; sys.path.insert(0, '.')
from agents.local_router import local_router
local_router.load()
p = local_router.find_path((13.0350, 77.5970), (12.9290, 77.5878))
print(f'\n{p[\'distance_m\']/1000:.2f} km, {p[\'travel_time_s\']/60:.1f} min, {len(p[\'edge_ids\'])} segments\n')
print('via', ' -> '.join(p[\'roads\'][:6]))
print(local_router.meta)
"
```

Real Bengaluru road names come back — Bellary Road, Mekhri Circle Underpass, Hosur Road. You cannot fake that with a lookup table.

Live traffic is genuinely live

The offline graph estimates Hebbal → Jayadeva at ~18 min. Google Routes with live traffic returns ~41 min for the same pair. That gap is real congestion, and it flows into every ETA and every hospital ranking.

Interactive API docs

`http://localhost:8000/docs` — every endpoint, every schema, try any of them live in front of the judges.

3. The endpoints, by role

Citizen — POST /api/incidents · GET /api/incidents/{id} **Dispatch** — GET /api/incidents/ · POST /api/incidents/{id}/approve **Field unit** — GET /api/driver/active · POST /api/driver/arrived_pickup · POST /api/driver/change_hospital **Hospital** — GET /api/hospital/{id}/incoming · PATCH /api/hospital/{id}/resources **Chat** — GET /api/chat/{id}/messages · POST /api/chat/{id}/send · WS /ws/chat/{id} **City model** — GET /api/city/state · POST /api/city/close · POST /api/city/conditions · GET /api/city/overlay · GET /api/city/explain/{edge} · POST /api/city/clear **Agents** — POST /api/fusion/combine · POST /api/hospital/rank · POST /api/route/compute · POST /api/traffic/analyse · POST /api/accessibility/report **Platform** — GET /health · GET /docs

4. Answers to the questions you will get

“Are those ICU bed numbers real?” No, and we say so on screen. Hospital *capability* — cath lab, stroke unit, trauma centre, blood bank — is curated for 30 Bengaluru facilities and is what drives routing. Live *availability* is modelled, because no hospital HMIS or HL7 feed is integrated. The two are drawn differently everywhere they appear.

“Is this causal AI?” It is a *structural causal model*, not causal inference. We specify the mechanisms rather than learning them, because we have no historical incident corpus to identify effects from. What that buys us is interventions and counterfactuals: in the real world you cannot run `do(close Bellary Road)`; inside a model we own, you can.

“Is the traffic model calibrated?” No. The coefficients are literature-informed estimates, not fitted to Bengaluru traffic counts. Directionally they are sound — closing an arterial raises congestion on parallel roads, rain slows everything, rush hour is worse than 3am — but the magnitudes are plausible rather than validated. That disclaimer is printed in the Disruptions panel and returned in the API response.

“What is the AI here — is it just an API call?” Four things the API does not do: a specialty-bypass gate that screens hospitals on clinical capability before proximity; a two-stage ranking that recomputes real road ETAs for the shortlist; an explainable fusion layer that states which option it rejected and why; and a causal city model where a closure displaces traffic onto neighbouring roads.

“What happens if Google goes down?” Beat 8. Turn off the wi-fi and watch.

“Is it secure?” No, and it does not pretend to be. Role selection grants the role, with no credential check, and the login screen says so. Real authentication and authorisation is the first thing a production deployment needs.

“What would you build next?” Calibrate the displacement coefficients against real traffic counts; integrate one hospital’s live bed feed to replace the modelled availability; and multi-incident resource contention, so two simultaneous calls compete for the same ambulance instead of each assuming an infinite fleet.

5. If something breaks mid-demo

Symptom	Cause	Fix
Console shows Backend offline	uvicorn died	restart it; the badge clears itself
Degraded data banner	Google Routes unreachable	expected — this is Beat 8, lean into it
Map tiles grey	OpenStreetMap rate limit	reload; tiles are cached after first load
Queue empty	incidents cleared	raise an SOS from Tab 1
Everything slow	first Overpass call	hospital catalogue caches for 15 min after the first fetch
Port 5173 busy	an old Vite still running	<code>Get-NetTCPConnection -LocalPort 5173 -State Listen</code> gives the PID

Record a backup video the night before. Venue wi-fi is the one failure the offline tier cannot save you from — if the laptop cannot reach the projector, no amount of graceful degradation helps.

6. The five sentences that matter

1. The nearest hospital is often the wrong one — a cardiac call needs a cath lab, not proximity.
2. We drove 11 minutes further and reached treatment 48 minutes sooner, because the nearer hospital would have had to transfer the patient onward.
3. Close one road and a different ambulance across the corridor gets slower — traffic displaces, it does not evaporate.
4. When live routing fails, it falls back to a real A* router over 112,725 OpenStreetMap nodes and labels itself degraded instead of guessing.
5. Every number on screen is either measured or marked as modelled — and we will show you which is which before you ask.